

Strings & Arrays Assignment

Instructions: Follow the same pattern as before —

1. Dry run each input on paper,
2. Write and test your code,
3. Submit both your dry run and final working code.

YENJOYY !!

1. Score of a String

You are given a string s . The score of a string is defined as the sum of the absolute difference between the ASCII values of adjacent characters.

Return the score of s .

Example 1:

Input: $s = \text{"hello"}$

Output: 13

Explanation:

The ASCII values of the characters in s are:

'h' = 104, 'e' = 101, 'l' = 108, 'o' = 111.

So, the score of s would be

$|104 - 101| + |101 - 108| + |108 - 108| + |108 - 111| = 3 + 7 + 0 + 3 = 13$.

Example 2:

Input: $s = \text{"zaz"}$

Output: 50

Explanation:

The ASCII values of the characters in s are:

'z' = 122, 'a' = 97

So, the score of s would be

$|122 - 97| + |97 - 122| = 25 + 25 = 50$.

Constraints:

- $2 \leq s.length \leq 100$
- s consists only of lowercase English letters.

2. Minimum Number of Operations to Move All Balls to Each Box

You have n boxes. You are given a binary string `boxes` of length n , where `boxes[i]` is '0' if the i th box is empty, and '1' if it contains one ball.

In one operation, you can move one ball from a box to an adjacent box. Box i is adjacent to box j if $\text{abs}(i - j) == 1$. Note that after doing so, there may be more than one ball in some boxes.

Return an array `answer` of size n , where `answer[i]` is the minimum number of operations needed to move all the balls to the i th box.

Each `answer[i]` is calculated considering the initial state of the boxes.

Example 1:

Input: `boxes = "110"`

Output: `[1,1,3]`

Explanation:

The answer for each box is as follows:

1. First box: you will have to move one ball from the second box to the first box in one operation.
2. Second box: you will have to move one ball from the first box to the second box in one operation.
3. Third box: you will have to move one ball from the first box to the third box in two operations, and move one ball from the second box to the third box in one operation.

Example 2:

Input: `boxes = "001011"`

Output: `[11,8,5,4,3,4]`

Constraints:

- $n == \text{boxes.length}$
- $1 \leq n \leq 2000$
- `boxes[i]` is either '0' or '1'.

3. Jewels and Stones

You're given strings `jewels` representing the types of stones that are jewels, and `stones` representing the stones you have. Each character in `stones` is a type of stone you have. You want to know how many of the stones you have are also jewels.

Letters are case sensitive, so "a" is considered a different type of stone from "A".

Example 1:

Input: `jewels = "aA"`, `stones = "aAAbbbb"`

Output: 3

Example 2:

Input: `jewels = "z"`, `stones = "ZZ"`

Output: 0

Constraints:

- $1 \leq \text{jewels.length}, \text{stones.length} \leq 50$
- `jewels` and `stones` consist of only English letters.
- All the characters of `jewels` are unique.

4. Maximum Number of Words Found in Sentences

A sentence is a list of words that are separated by a single space with no leading or trailing spaces.

You are given an array of strings `sentences`, where each `sentences[i]` represents a single sentence.

Return the maximum number of words that appear in a single sentence.

Example 1:

Input: `sentences = ["alice and bob love leetcode", "i think so too", "this is great thanks very much"]`

Output: 6

Explanation:

- The first sentence, "alice and bob love leetcode", has 5 words in total.
- The second sentence, "i think so too", has 4 words in total.
- The third sentence, "this is great thanks very much", has 6 words in total.
Thus, the maximum number of words in a single sentence comes from the third sentence, which has 6 words.

Example 2:

Input: sentences = ["please wait", "continue to fight", "continue to win"]

Output: 3

Explanation: It is possible that multiple sentences contain the same number of words.

In this example, the second and third sentences have the same number of words.

Constraints:

- $1 \leq \text{sentences.length} \leq 100$
- $1 \leq \text{sentences}[i].\text{length} \leq 100$
- `sentences[i]` consists only of lowercase English letters and ' ' only.
- `sentences[i]` does not have leading or trailing spaces.
- All the words in `sentences[i]` are separated by a single space.

5. Truncate Sentence

A sentence is a list of words that are separated by a single space with no leading or trailing spaces.

Each of the words consists of only uppercase and lowercase English letters (no punctuation).

For example, "Hello World", "HELLO", and "hello world hello world" are all sentences.

You are given a sentence **s** and an integer **k**. You want to truncate **s** such that it contains only the first **k** words. Return **s** after truncating it.

Example 1:

Input: s = "Hello how are you Contestant", k = 4

Output: "Hello how are you"

Explanation:

The words in **s** are ["Hello", "how", "are", "you", "Contestant"].

The first 4 words are ["Hello", "how", "are", "you"].

Hence, you should return "Hello how are you".

Example 2:

Input: s = "What is the solution to this problem", k = 4

Output: "What is the solution"

Explanation:

The words in **s** are ["What", "is", "the", "solution", "to", "this", "problem"].

The first 4 words are ["What", "is", "the", "solution"].

Hence, you should return "What is the solution".

Example 3:

Input: `s = "chopper is not a tanuki"`, `k = 5`

Output: `"chopper is not a tanuki"`

Constraints:

- `1 <= s.length <= 500`
- `k` is in the range `[1, the number of words in s]`.
- `s` consists of only lowercase and uppercase English letters and spaces.
- The words in `s` are separated by a single space.
- There are no leading or trailing spaces.

6. Find First Palindromic String in the Array

Given an array of strings `words`, return the first palindromic string in the array. If there is no such string, return an empty string `" "`.

A string is palindromic if it reads the same forward and backward.

Example 1:

Input: `words = ["abc", "car", "ada", "racecar", "cool"]`

Output: `"ada"`

Explanation: The first string that is palindromic is `"ada"`.

Note that `"racecar"` is also palindromic, but it is not the first.

Example 2:

Input: `words = ["notapalindrome", "racecar"]`

Output: `"racecar"`

Explanation: The first and only string that is palindromic is `"racecar"`.

Example 3:

Input: `words = ["def", "ghi"]`

Output: `" "`

Explanation: There are no palindromic strings, so the empty string is returned.

Constraints:

- `1 <= words.length <= 100`

- $1 \leq \text{words}[i].\text{length} \leq 100$
- `words[i]` consists only of lowercase English letters.

7. Check if the Sentence Is Pangram

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string `sentence` containing only lowercase English letters, return `true` if `sentence` is a pangram, or `false` otherwise.

Example 1:

Input: `sentence = "thequickbrownfoxjumpsoverthelazydog"`

Output: `true`

Explanation: `sentence` contains at least one of every letter of the English alphabet.

Example 2:

Input: `sentence = "leetcode"`

Output: `false`

Constraints:

$1 \leq \text{sentence.length} \leq 1000$

`sentence` consists of lowercase English letters.

8. Reverse Words in a String

Given a string `s`, reverse the order of characters in each word within a sentence while still preserving whitespace and initial word order.

Example 1:

Input: `s = "Let's take LeetCode contest"`

Output: `"s'teL ekat edoCteeL tsetnoc"`

Example 2:

Input: `s = "Mr Ding"`

Output: `"rM gniD"`

Constraints:

$1 \leq \text{s.length} \leq 5 * 10^4$

s contains printable ASCII characters.
s does not contain any leading or trailing spaces.
There is at least one word in s.
All the words in s are separated by a single space.

9. Number of Strings That Appear as Substrings in Word

Given an array of strings patterns and a string word, return the number of strings in patterns that exist as a substring in word.

A substring is a contiguous sequence of characters within a string.

Example 1:

Input: patterns = ["a","abc","bc","d"], word = "abc"

Output: 3

Explanation:

- "a" appears as a substring in "abc".
 - "abc" appears as a substring in "abc".
 - "bc" appears as a substring in "abc".
 - "d" does not appear as a substring in "abc".
- 3 of the strings in patterns appear as a substring in word.

Example 2:

Input: patterns = ["a","b","c"], word = "aaaaabbbbb"

Output: 2

Explanation:

- "a" appears as a substring in "aaaaabbbbb".
 - "b" appears as a substring in "aaaaabbbbb".
 - "c" does not appear as a substring in "aaaaabbbbb".
- 2 of the strings in patterns appear as a substring in word.

Example 3:

Input: patterns = ["a","a","a"], word = "ab"

Output: 3

Explanation: Each of the patterns appears as a substring in word "ab".

Constraints:

1 <= patterns.length <= 100

1 <= patterns[i].length <= 100
1 <= word.length <= 100
patterns[i] and word consist of lowercase English letters.

10. Determine if String Halves Are Alike

You are given a string `s` of even length. Split this string into two halves of equal lengths, and let `a` be the first half and `b` be the second half.

Two strings are alike if they have the same number of vowels ('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'). Notice that `s` contains uppercase and lowercase letters.

Return `true` if `a` and `b` are alike. Otherwise, return `false`.

Example 1:

Input: `s = "book"`

Output: `true`

Explanation: `a = "bo"` and `b = "ok"`. `a` has 1 vowel and `b` has 1 vowel. Therefore, they are alike.

Example 2:

Input: `s = "textbook"`

Output: `false`

Explanation: `a = "text"` and `b = "book"`. `a` has 1 vowel whereas `b` has 2. Therefore, they are not alike. Notice that the vowel `o` is counted twice.

Constraints:

2 <= `s.length` <= 1000

`s.length` is even.

`s` consists of uppercase and lowercase letters.

11. Remove Trailing Zeros From a String

You are given a positive integer `num` represented as a string. Return the integer `num` without trailing zeros as a string.

Example 1:

Input: `num = "51230100"`

Output: `"512301"`

Explanation: Integer `"51230100"` has 2 trailing zeros, remove them and return `"512301"`.

Example 2:

Input: num = "123"

Output: "123"

Explanation: Integer "123" has no trailing zeros, return it as is.

Constraints:

- $1 \leq \text{num.length} \leq 1000$
- num consists of only digits
- num doesn't have any leading zeros

12. Find Common Characters

Given a string array **words**, return an array of all characters that show up in all strings within the **words** (including duplicates). You may return the answer in any order.

Example 1:

Input: words = ["bella", "label", "roller"]

Output: ["e", "l", "l"]

Example 2:

Input: words = ["cool", "lock", "cook"]

Output: ["c", "o"]

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $1 \leq \text{words}[i].\text{length} \leq 100$
- words[i] consists of lowercase English letters.

13. Delete Characters to Make Fancy String

A fancy string is a string where no three consecutive characters are equal.

Given a string **s**, delete the minimum possible number of characters from **s** to make it fancy.

Return the final string after the deletion. It can be shown that the answer will always be unique.

Example 1:

Input: `s = "leeetcode"`

Output: `"leetcode"`

Explanation:

Remove an 'e' from the first group of 'e's to create "leetcode".

No three consecutive characters are equal, so return "leetcode".

Example 2:

Input: `s = "aaabaaaa"`

Output: `"aabaa"`

Explanation:

Remove an 'a' from the first group of 'a's to create "aabaaaa".

Remove two 'a's from the second group of 'a's to create "aabaa".

No three consecutive characters are equal, so return "aabaa".

Example 3:

Input: `s = "aab"`

Output: `"aab"`

Explanation: No three consecutive characters are equal, so return "aab".

Constraints:

- $1 \leq s.length \leq 10^5$
- `s` consists only of lowercase English letters.

14. Destination City

You are given the array `paths`, where `paths[i] = [cityAi, cityBi]` means there exists a direct path going from `cityAi` to `cityBi`. Return the destination city, that is, the city without any path outgoing to another city.

It is guaranteed that the graph of paths forms a line without any loop, therefore, there will be exactly one destination city.

Example 1:

Input: `paths = [["London", "New York"], ["New York", "Lima"], ["Lima", "Sao Paulo"]]`

Output: `"Sao Paulo"`

Explanation: Starting at "London" city you will reach "Sao Paulo" city which is the destination city.

Your trip consists of: "London" -> "New York" -> "Lima" -> "Sao Paulo".

Example 2:

Input: `paths = [["B", "C"], ["D", "B"], ["C", "A"]]`

Output: `"A"`

Explanation: All possible trips are:

"D" -> "B" -> "C" -> "A".

"B" -> "C" -> "A".

"C" -> "A".

"A".

Clearly the destination city is "A".

Example 3:

Input: paths = [["A","Z"]]

Output: "Z"

Constraints:

- $1 \leq \text{paths.length} \leq 100$
- $\text{paths}[i].\text{length} == 2$
- $1 \leq \text{cityAi.length}, \text{cityBi.length} \leq 10$
- $\text{cityAi} \neq \text{cityBi}$
- All strings consist of lowercase and uppercase English letters and the space character.

15. Flip Columns For Maximum Number of Equal Rows

You are given an $m \times n$ binary matrix `matrix`.

You can choose any number of columns in the matrix and flip every cell in that column (i.e., change the value of the cell from 0 to 1 or vice versa).

Return the maximum number of rows that have all values equal after some number of flips.

Example 1:

Input: `matrix = [[0,1],[1,1]]`

Output: 1

Explanation: After flipping no values, 1 row has all values equal.

Example 2:

Input: `matrix = [[0,1],[1,0]]`

Output: 2

Explanation: After flipping values in the first column, both rows have equal values.

Example 3:

Input: `matrix = [[0,0,0],[0,0,1],[1,1,0]]`

Output: 2

Explanation: After flipping values in the first two columns, the last two rows have equal values.

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 300$

- `matrix[i][j]` is either 0 or 1.

16. Sort Array By Parity

Given an integer array `nums`, move all the even integers to the beginning of the array followed by all the odd integers.

Return any array that satisfies this condition.

Example 1:

Input: `nums = [3,1,2,4]`

Output: `[2,4,3,1]`

Explanation: The outputs `[4,2,3,1]`, `[2,4,1,3]`, and `[4,2,1,3]` would also be accepted.

Example 2:

Input: `nums = [0]`

Output: `[0]`

Constraints:

- `1 <= nums.length <= 5000`
- `0 <= nums[i] <= 5000`

17. Minimum Limit of Balls in a Bag

You are given an integer array `nums` where the *i*th bag contains `nums[i]` balls. You are also given an integer `maxOperations`.

You can perform the following operation at most `maxOperations` times:

- Take any bag of balls and divide it into two new bags with a positive number of balls. For example, a bag of 5 balls can become two new bags of 1 and 4 balls, or two new bags of 2 and 3 balls.

Your penalty is the maximum number of balls in a bag. You want to minimize your penalty after the operations.

Return the minimum possible penalty after performing the operations.

Example 1:

Input: `nums = [9], maxOperations = 2`

Output: 3

Explanation:

- Divide the bag with 9 balls into two bags of sizes 6 and 3. `[9] -> [6,3]`.

- Divide the bag with 6 balls into two bags of sizes 3 and 3. $[6,3] \rightarrow [3,3,3]$.
The bag with the most number of balls has 3 balls, so your penalty is 3 and you should return 3.

Example 2:

Input: `nums = [2,4,8,2]`, `maxOperations = 4`

Output: 2

Explanation:

- Divide the bag with 8 balls into two bags of sizes 4 and 4. $[2,4,8,2] \rightarrow [2,4,4,4,2]$.
- Divide the bag with 4 balls into two bags of sizes 2 and 2. $[2,4,4,4,2] \rightarrow [2,2,2,4,4,2]$.
- Divide the bag with 4 balls into two bags of sizes 2 and 2. $[2,2,2,4,4,2] \rightarrow [2,2,2,2,2,4,2]$.
- Divide the bag with 4 balls into two bags of sizes 2 and 2. $[2,2,2,2,2,4,2] \rightarrow [2,2,2,2,2,2,2,2]$.
The bag with the most number of balls has 2 balls, so your penalty is 2, and you should return 2.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{maxOperations}, \text{nums}[i] \leq 10^9$

18. Sign of the Product of an Array

Implement a function `signFunc(x)` that returns:

- 1 if x is positive.
- -1 if x is negative.
- 0 if x is equal to 0.

You are given an integer array `nums`. Let `product` be the product of all values in the array `nums`.

Return `signFunc(product)`.

Example 1:

Input: `nums = [-1,-2,-3,-4,3,2,1]`

Output: 1

Explanation: The product of all values in the array is 144, and `signFunc(144) = 1`

Example 2:

Input: `nums = [1,5,0,2,-3]`

Output: 0

Explanation: The product of all values in the array is 0, and `signFunc(0) = 0`

Example 3:

Input: `nums = [-1,1,-1,1,-1]`

Output: -1

Explanation: The product of all values in the array is -1, and `signFunc(-1) = -1`

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $-100 \leq \text{nums}[i] \leq 100$

19. Corporate Flight Bookings

There are n flights labeled from 1 to n .

You are given an array of flight bookings `bookings`, where `bookings[i] = [firsti, lasti, seatsi]` represents a booking for flights `firsti` through `lasti` (inclusive) with `seatsi` seats reserved for each flight in that range.

Return an array `answer` of length n , where `answer[i]` is the total number of seats reserved for flight $i + 1$.

Example 1:

Input: `bookings = [[1,2,10],[2,3,20],[2,5,25]]`, $n = 5$

Output: `[10,55,45,25,25]`

Explanation:

Flight labels: 1 2 3 4 5

Booking 1 reserved: 10 10

Booking 2 reserved: 20 20

Booking 3 reserved: 25 25 25 25

Total seats: 10 55 45 25 25

Hence, `answer = [10,55,45,25,25]`

Example 2:

Input: `bookings = [[1,2,10],[2,2,15]]`, $n = 2$

Output: `[10,25]`

Explanation:

Flight labels: 1 2

Booking 1 reserved: 10 10

Booking 2 reserved: 15

Total seats: 10 25
Hence, answer = [10,25]

Constraints:

- $1 \leq n \leq 2 * 10^4$
- $1 \leq \text{bookings.length} \leq 2 * 10^4$
- $\text{bookings}[i].\text{length} == 3$
- $1 \leq \text{first}_i \leq \text{last}_i \leq n$
- $1 \leq \text{seats}_i \leq 10^4$

20. Rabbits in Forest

There is a forest with an unknown number of rabbits. We asked n rabbits "How many rabbits have the same color as you?" and collected the answers in an integer array **answers** where **answers[i]** is the answer of the i th rabbit.

Given the array **answers**, return the minimum number of rabbits that could be in the forest.

Example 1:

Input: answers = [1,1,2]

Output: 5

Explanation:

The two rabbits that answered "1" could both be the same color, say red.

The rabbit that answered "2" can't be red or the answers would be inconsistent.

Say the rabbit that answered "2" was blue.

Then there should be 2 other blue rabbits in the forest that didn't answer in the array.

The smallest possible number of rabbits in the forest is therefore 5: 3 that answered plus 2 that didn't.

Example 2:

Input: answers = [10,10,10]

Output: 11

Constraints:

- $1 \leq \text{answers.length} \leq 1000$
- $0 \leq \text{answers}[i] < 1000$

21. Count of Interesting Subarrays

You are given a 0-indexed integer array `nums`, an integer `modulo`, and an integer `k`.

Your task is to find the count of subarrays that are interesting.

A subarray `nums[l..r]` is interesting if the following condition holds:

Let `cnt` be the number of indices `i` in the range `[l, r]` such that `nums[i] % modulo == k`. Then, `cnt % modulo == k`.

Return an integer denoting the count of interesting subarrays.

Note: A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [3,2,4]`, `modulo = 2`, `k = 1`

Output: 3

Example 2:

Input: `nums = [3,1,9,6]`, `modulo = 3`, `k = 0`

Output: 2

Constraints:

$1 \leq \text{nums.length} \leq 10^5$

$1 \leq \text{nums}[i] \leq 10^9$

$1 \leq \text{modulo} \leq 10^9$

$0 \leq k < \text{modulo}$

22. Verifying an Alien Dictionary

In an alien language, surprisingly, they also use English lowercase letters, but possibly in a different order. The order of the alphabet is some permutation of lowercase letters.

Given a sequence of words written in the alien language, and the order of the alphabet, return true if and only if the given words are sorted lexicographically in this alien language.

Example 1:

Input: `words = ["hello", "leetcode"]`, `order = "hlabcdefghijklmnopqrstuvwxyz"`

Output: true

Example 2:

Input: `words = ["word", "world", "row"]`, `order = "worldabcefghijklmnopqstuvxyz"`

Output: false

Example 3:

Input: words = ["apple", "app"], order = "abcdefghijklmnopqrstuvwxyz"

Output: false

Constraints:

$1 \leq \text{words.length} \leq 100$

$1 \leq \text{words}[i].\text{length} \leq 20$

$\text{order.length} == 26$

All characters in words[i] and order are English lowercase letters.

23. Matrix Block Sum

Given a $m \times n$ matrix mat and an integer k, return a matrix answer where each answer[i][j] is the sum of all elements mat[r][c] for:

- $i - k \leq r \leq i + k$,
- $j - k \leq c \leq j + k$, and
- (r, c) is a valid position in the matrix.

Example 1:

Input: mat = [[1,2,3],[4,5,6],[7,8,9]], k = 1

Output: [[12,21,16],[27,45,33],[24,39,28]]

Example 2:

Input: mat = [[1,2,3],[4,5,6],[7,8,9]], k = 2

Output: [[45,45,45],[45,45,45],[45,45,45]]

Constraints:

$m == \text{mat.length}$

$n == \text{mat}[i].\text{length}$

$1 \leq m, n, k \leq 100$

$1 \leq \text{mat}[i][j] \leq 100$